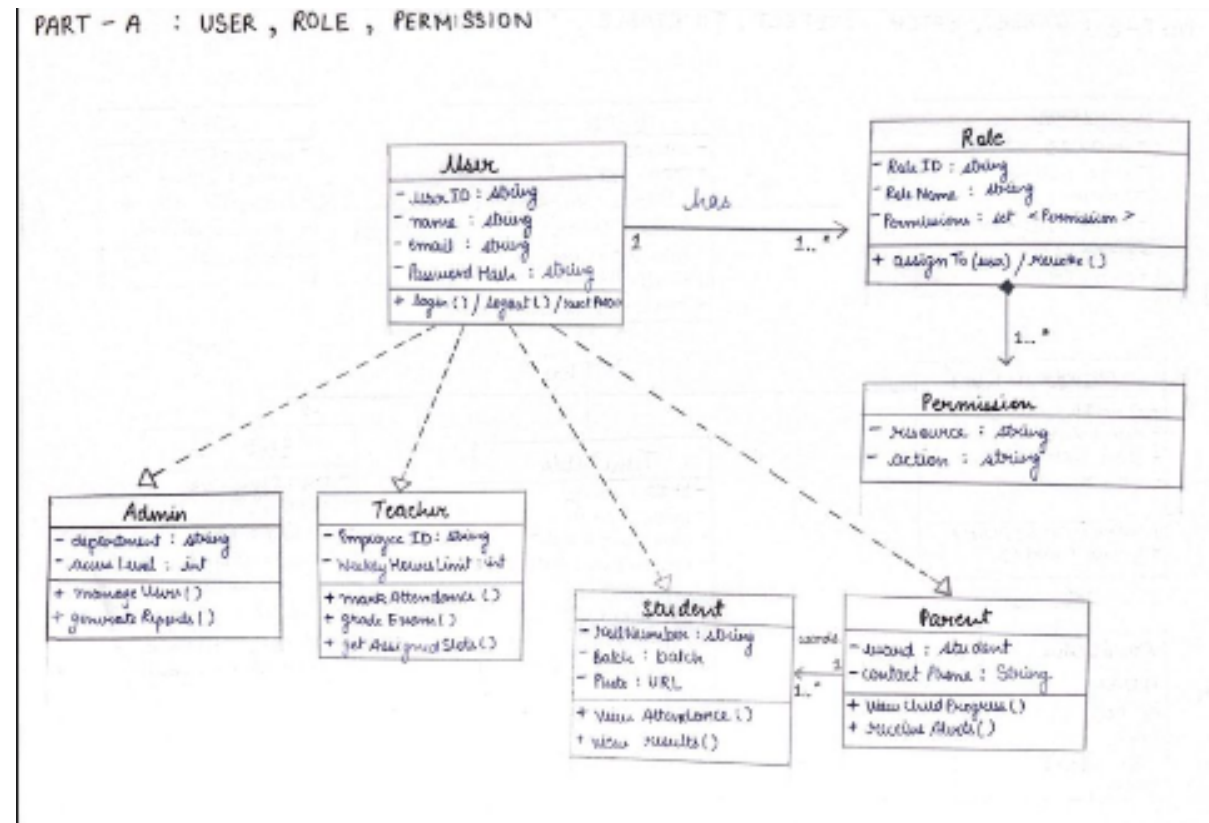


## Task 1: System Design & Report

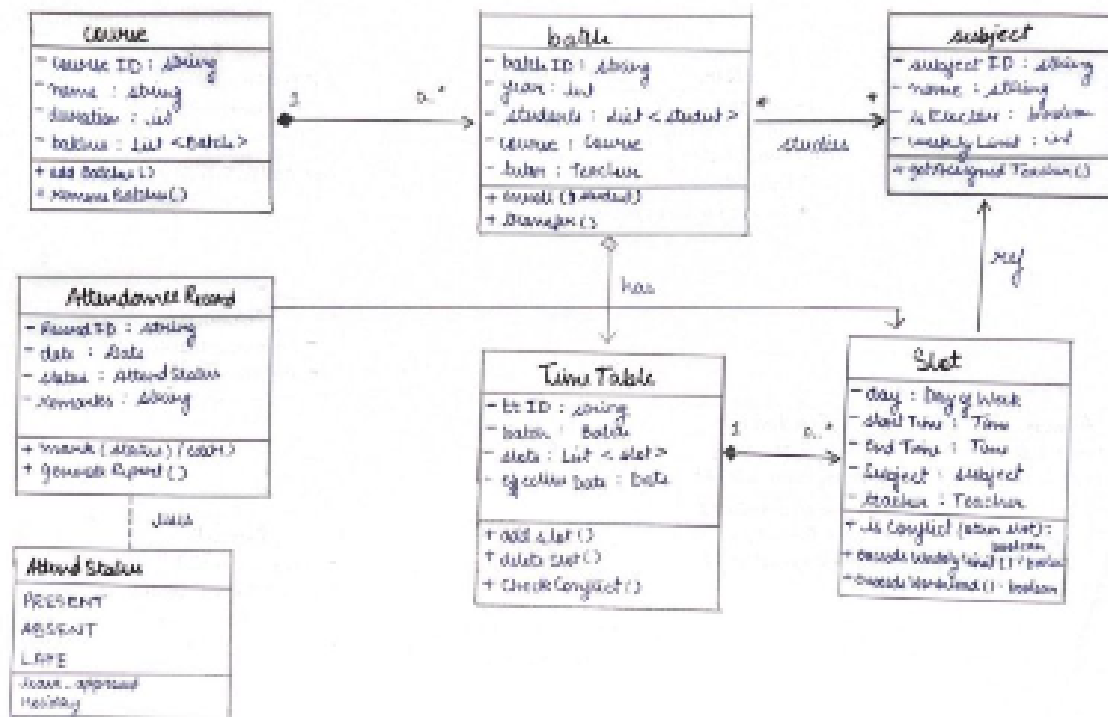
Tanishi Tyagi 2024111027

Shravan Kannan 2024117003

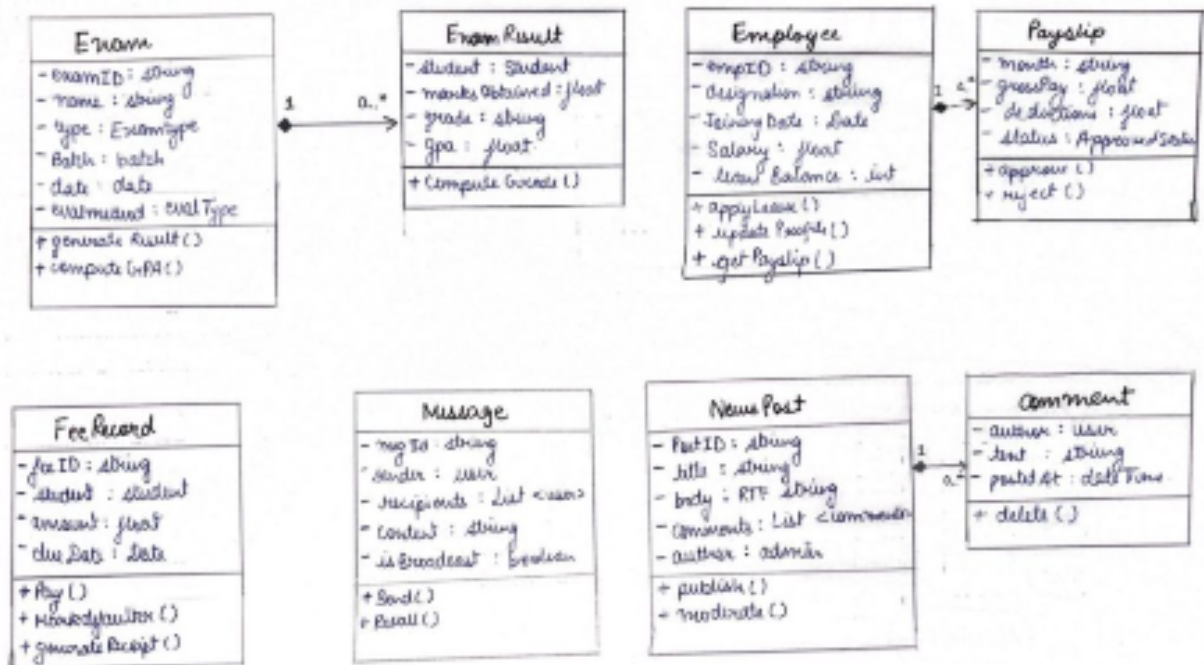
### 1. Hand-drawn UML Class Diagram

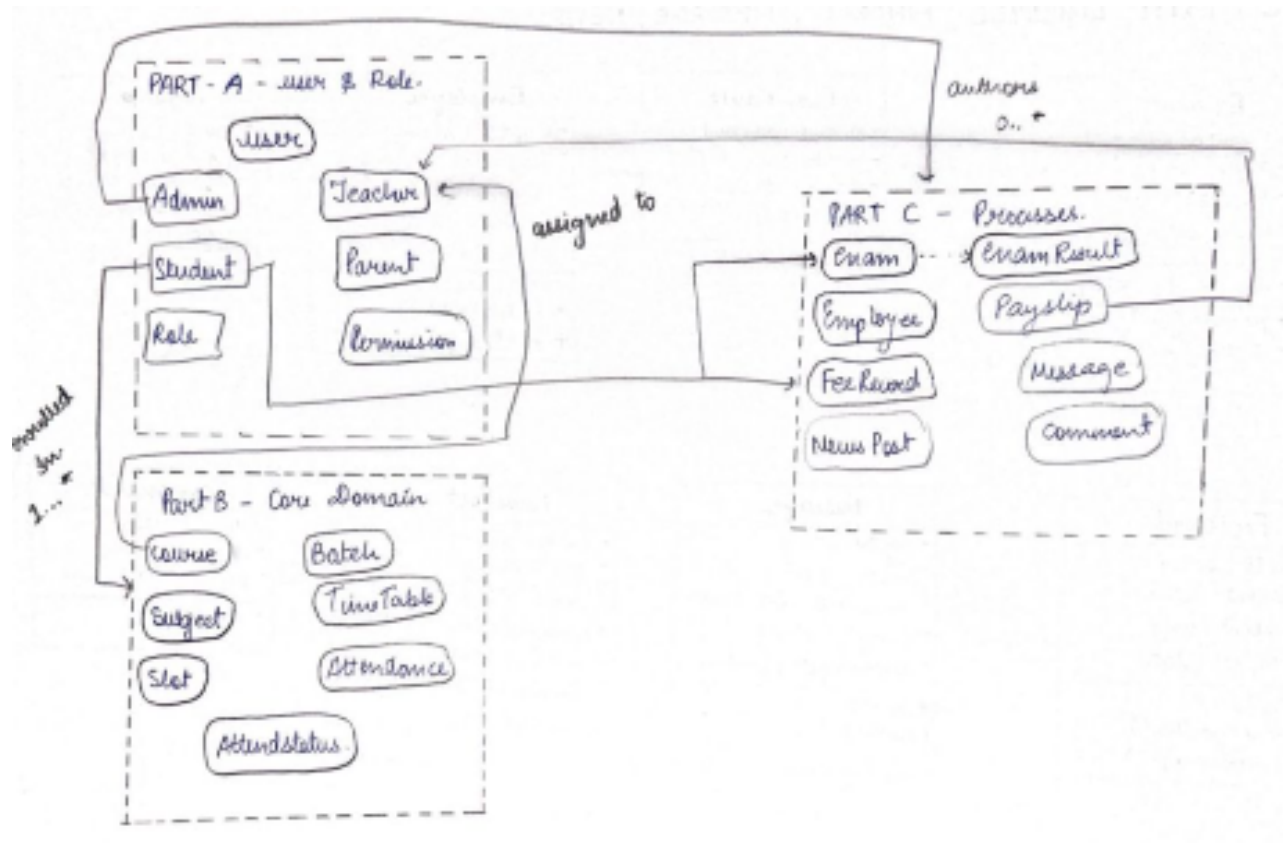


## PART - B : COURSE , BATCH , SUBJECT , TIMETABLE , ATTENDANCE.



## PART - C : EXAM , EMPLOYEE , FINANCE , MESSAGE , NEWS.





## 2. Responsibilities Table

| Class Name                                | Core Responsibility                                                |
|-------------------------------------------|--------------------------------------------------------------------|
| <b>SystemConfiguration</b>                | Stores and manages global institute settings (currency, timezone). |
| <b>User (Abstract)</b>                    | Handles base authentication (login/logout) and identity data.      |
| <b>Admin / Teacher / Student / Parent</b> | Represents specific user roles with dedicated access privileges.   |

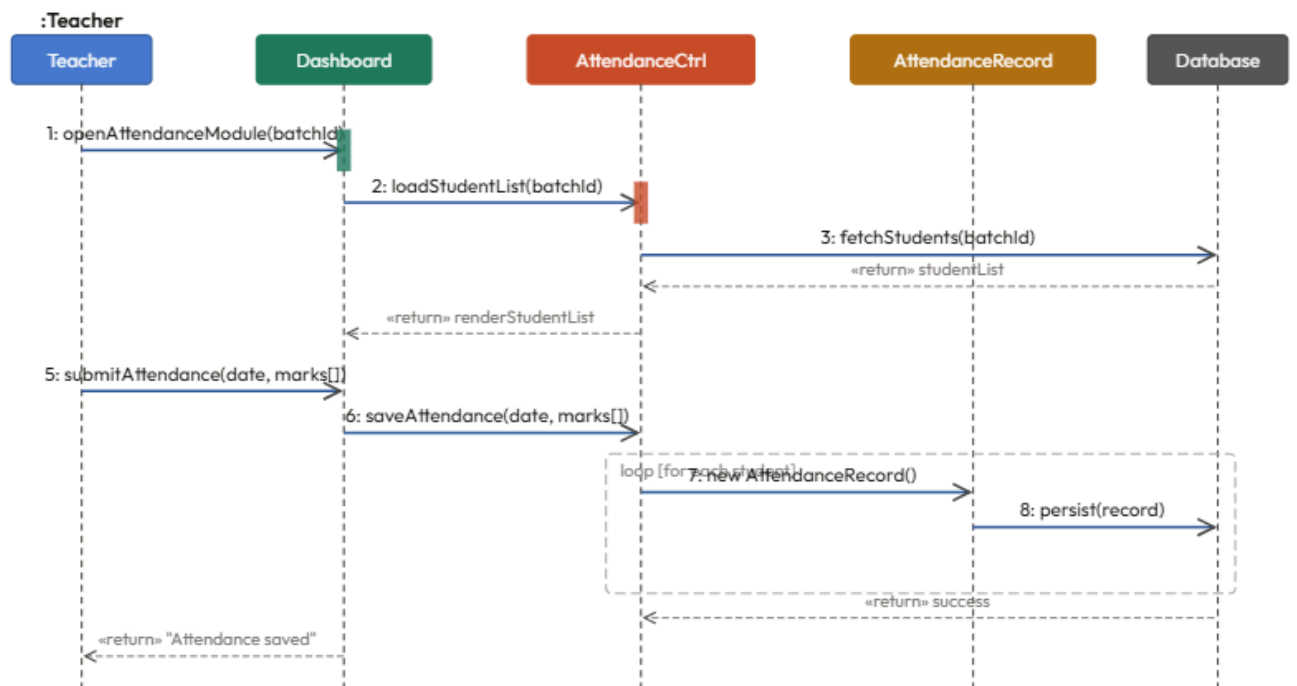
|                               |                                                                      |
|-------------------------------|----------------------------------------------------------------------|
| <b>Course</b>                 | Defines an academic program and manages its associated batches.      |
| <b>Batch</b>                  | Groups enrolled students for a specific course year.                 |
| <b>TimeTable</b>              | Organizes the weekly schedule for a specific batch.                  |
| <b>Slot</b>                   | Defines a specific time period, subject, and assigned teacher.       |
| <b>AttendanceRecord</b>       | Tracks the presence, absence, or leave status of a specific student. |
| <b>Employee</b>               | Manages payroll, designation, and leave request tracking.            |
| <b>Transaction (Abstract)</b> | Serves as the base for logging all financial movements.              |
| <b>FeeRecord</b>              | Tracks student payments, amounts, and due dates.                     |
| <b>Exam</b>                   | Defines the evaluation parameters and grading methods for a batch.   |

---

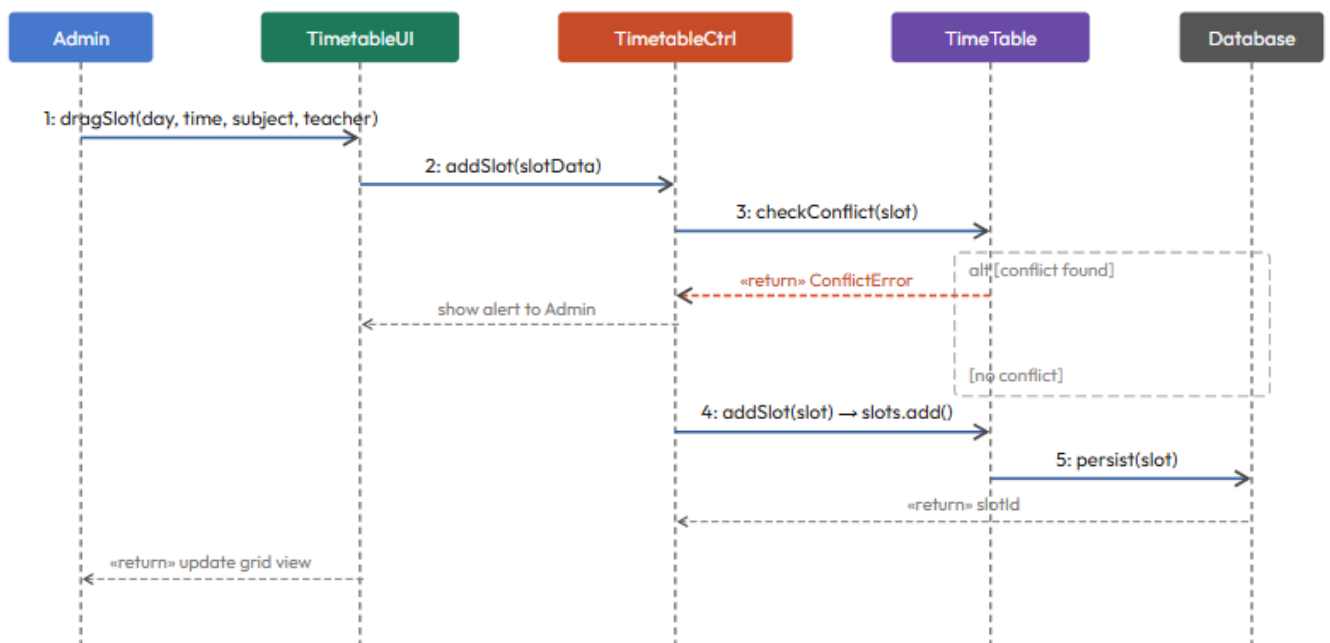
### 3. Dynamic Diagrams

### 3.1 Sequence Diagrams

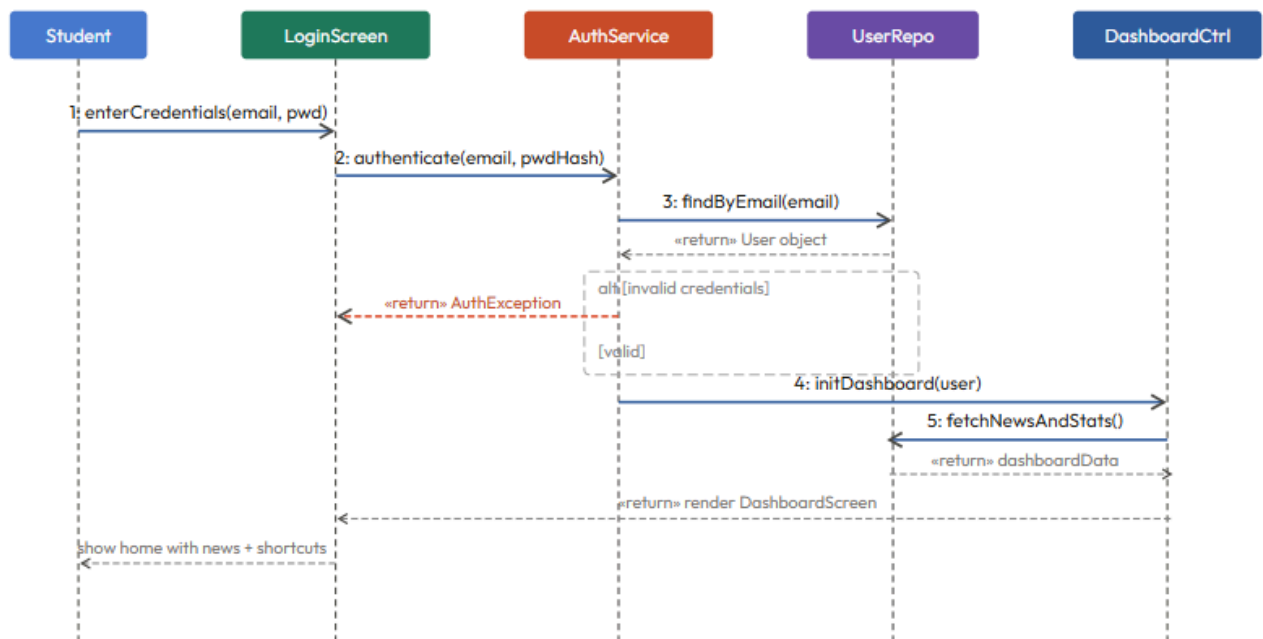
#### 1. Sequence 1- Mark attendance (Teacher Marks a class)



#### 2. Sequence 2- Add a slot to the timetable (Admin)

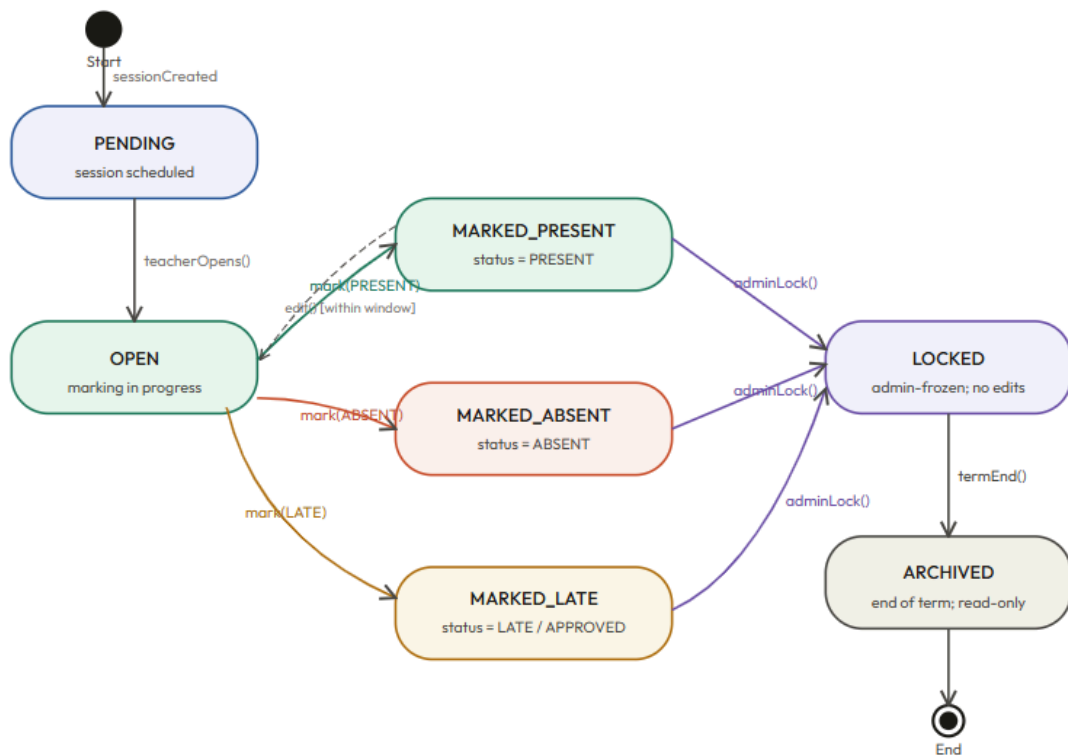


### 3. Sequence 3- Student Login and dashboard navigation



### 3.2 State Diagram

- Attendance Record



---

## 4. Design Narrative & Analysis

- **Low Coupling:** The system is divided into independent modules. For example, the `Finance` module interacts with the `Student` class via abstract IDs, meaning changes in the finance logic do not break the core academic models.
  - **High Cohesion:** Each class has a single, focused purpose. The `AttendanceRecord` exclusively handles attendance data, and `Slot` strictly handles scheduling, keeping related data tightly bound.
  - **Separation of Concerns:** The architecture clearly separates User Management, Core Academic Domain (Batches/Courses), and Administrative Processes (HR/Finance) into distinct logical boundaries.
  - **Law of Demeter:** Objects only interact with their direct dependencies. A `Student` object can query its enrolled `Batch`, but it does not reach through the `Batch` to directly manipulate the parent `Course` object.
- 

## 5. Pattern & Anti-Pattern Analysis

### Design Patterns Present:

1. **Singleton Pattern:** Applied to the `SystemConfiguration` class to ensure only one global instance manages the institute's shared settings (language, timezone).
2. **Strategy Pattern:** Implicitly used in the `Exam` class via the `EvalType` attribute. This allows the system to swap out grading algorithms (GPA, CCE, CWA) dynamically without altering the core exam logic.

### Design Anti-Patterns Avoided:

1. **God Object (The Blob):** Avoided creating a massive, centralized `InstituteManager` class that controls everything. Responsibilities are strictly delegated to smaller, modular classes.
  2. **Poltergeist (Useless Classes):** The design actively eliminates redundant, intermediate controller classes that do nothing but pass data back and forth. Specifically, unnecessary abstractions like duplicate user eligibility nodes were removed to keep the hierarchy flat and efficient.
- 

## 6. Reflection

### Strongest Aspects:

1. **Extensibility:** The use of abstract base classes (`User`, `Transaction`) makes it incredibly easy to add new roles or financial tracking types in the future without modifying existing code.

2. **Clear Domain Boundaries:** The modular split makes team implementation straightforward. The Attendance and Timetable modules can be built entirely independently of HR or Finance.

**Weakest Aspects:**

1. **Search Complexity:** The requirement for a "global dashboard search" across all modules is difficult to model purely in a static class diagram, as it requires indexing across highly decoupled classes.
2. **Reporting Overhead:** Generating comprehensive, cross-module reports (e.g., tying student attendance to fee defaults) requires complex data aggregation, which could lead to performance bottlenecks if the database isn't optimized.